

Python For Hackers

index here

- [f-string](#)
- [api fuzzer](#)
- [encrypt & decrypt](#)
- [dictionaries](#)
- [web basics HTML](#)
- [Spider n probe \(scraper\)](#)
- [Port Scanner](#)
- [if _name_=='_main_':](#)
- [Open Redirect](#)
- [IDORS](#)

English:

<https://chatgpt.com/share/6a7d8e7c-7f28-83ea-86c6-8eaa1455b37f>

Syntax:

f-string

f-string = string + Python expression interpolation(插入语)

f string:Put **f** before the string, then use **{ }** around the variable:

```
username = "admin"
password = "123456"

url = f"http://example.com/login?username={username}&password={password}"

print(url)
```

no need to concatenate.

You can put expressions inside `{}`

```
pi = 3.1415926

print(f"{pi:.2f}") ## keep 2 digits after the decimal point.
```

Very important: `{}` isn't just for variables, you can put Python expressions inside

```
print(f"{1 + 2}")
print(f"{10 * 5}")
print(f"{'hello'}.upper()")
```

Output:

```
3
50
HELLO
```

```
x = "5"
y = 7
print(int(x) + y)
```

Only variables of the same type can be operated on

```
def function(params);  
    xxx
```

```
time.sleep(5)
```

```
input("xxx").lower
```

api fuzzer

json api <https://chatgpt.com/share/6a8d8179-4a00-83e8-8aac-505e73092f96>

get.json endpoints <https://chatgpt.com/share/6a8d9a03-ddd0-83ee-9e06-e76a6e4a5858>

sys.stdin <https://chatgpt.com/share/6a8d9dc8-8e2c-83ee-844a-2ded9394bd9a>

```
import requests  
import sys  
  
def loop():  
    for word in sys.stdin:  
        res = requests.get(url=f"http://127.0.0.1/{word}") # domain  
        if res.status_code == 404:  
            continue  
        else:  
            data = res.json()  
            print(data)  
            print(res.status_code)  
            print(word)  
  
loop()
```

copy to kali

gedit xxx.py (edit)

cat xx.txt | python3 xxx.py (xx -> directory)

```
def xxx():  
    pass
```

encrypt & decrypt

```
import base64  
def encrypt_pass(password):  
    encoded_bytes = base64.b64encode(password.encode())  
    print(encoded_bytes)  
  
user_pass = input("Enter ur pswd: ")    encrypt_pass(user_pass)
```

```
import base64  
def decrypt_pass(password):  
    decoded_bytes = base64.b64decode(password)  
    decoded_data = decoded_bytes.decode()  
    print(f"decoded pswd is {decoded_data}")  
  
encode_string = input("enter the base64 string: ")  
decrypt_pass(encode_string)
```

why .decode() <https://chatgpt.com/share/6a8db296-19ec-83e8-b7c7-95fb0f1f4a95>

dictionaries

```
dic = {"key1": "value1", "key2": "value2"}
print(dic["key"]) # key

dic["key3"] = value3 # add element

dic = {} # empty it

for thing in dic:
    print(thing)
    print(dic[thing]) # loop through
```

web basics HTML

! and enter ->boiler plate 标准格式

```
<!-- Document structure -->
<!DOCTYPE html>

<!-- Root element of an HTML page -->
<html>
</html>

<!-- Contains information about the webpage -->
<head>
</head>

<!-- Sets the title shown in the browser tab -->
<title>This is a title</title>

<!-- Contains visible content of the webpage -->
<body>
</body>

<!-- Text elements -->

<!-- Main heading -->
<h1>This is the largest heading</h1>
```

```
<!-- Sub heading -->
<h2>This is a smaller heading</h2>
```

```
<!-- Paragraph -->
<p>This is a paragraph of text.</p>
```

```
<!-- Line break -->
<br>
```

```
<!-- Horizontal line -->
<hr size="10">
```

```
<!-- Text formatting -->
```

```
<!-- Bold text -->
<b>This is bold text</b>
```

```
<!-- Important text (usually bold) -->
<strong>This is important text</strong>
```

```
<!-- Italic text -->
<i>This is italic text</i>
```

```
<!-- Emphasized text -->
<em>This text is emphasized</em>
```

```
<!-- Underlined text -->
<u>This text is underlined</u>
```

```
<!-- Highlighted text -->
<mark>This text is highlighted</mark>
```

```
<!-- Deleted text -->
<del>This text is deleted</del>
```

```
<!-- Links and media -->
```

```
<!-- Hyperlink -->
<a href="https://example.com">Click here</a>

<a href="http://xxx.com">
  
</a>

<!-- Image -->


<!-- video -->
<video src="video.mp4" controls>
  Your browser does not support video.
</video>

<!-- Audio -->
<audio src="music.mp3" controls>
  Your browser does not support audio.
</audio>

<!-- Lists -->

<!-- Unordered list -->
<ul>
  <li>Apple</li>
  <li>Banana</li>
</ul>

<!-- Ordered list -->
<ol>
  <li>First item</li>
  <li>Second item</li>
</ol>

<!-- List item -->
<li>This is a list item</li>

<!-- Tables -->
```

```
<!-- Table -->
```

```
<table>
```

```
</table>
```

```
<!-- Table row -->
```

```
<tr>
```

```
</tr>
```

```
<!-- Table header -->
```

```
<th>Name</th>
```

```
<!-- Table data -->
```

```
<td>John</td>
```

```
<!-- Forms -->
```

```
<!-- Form container -->
```

```
<form class="" action="/submit" method="">
```

```
  <label>xxx</label>
```

```
</form>
```

```
<!-- Input box -->
```

```
<input type="text" placeholder="Enter your name">
```

```
<!-- Password input -->
```

```
<input type="password" placeholder="Password">
```

```
<!-- Checkbox -->
```

```
<input type="checkbox"> Accept terms
```

```
<!-- Radio button -->
```

```
<input type="radio"> Option A
```

```
<!-- Submit button -->
```

```
<input type="submit" value="Submit">
```

```
<!-- Button -->
```

```
<button>Click me</button>
```

```
<!-- Text area -->
```

```
<textarea>
```

```
Write something here
```

```
</textarea>
```

```
<!-- Drop-down menu -->
```

```
<select>
```

```
  <option>Option 1</option>
```

```
  <option>Option 2</option>
```

```
</select>
```

```
<!-- Semantic layout tags -->
```

```
<!-- Header section -->
```

```
<header>
```

```
  Website Header
```

```
</header>
```

```
<!-- Navigation menu -->
```

```
<nav>
```

```
  Navigation Links
```

```
</nav>
```

```
<!-- Main content -->
```

```
<main>
```

```
  Main Content
```

```
</main>
```

```
<!-- Section of content -->
```

```
<section>
```

```
  A Section
```

```
</section>
```

```
<!-- Independent article -->
```

```
<article>
```

```
  An Article
```

```
</article>
```

```
<!-- Sidebar content -->
<aside>
  Sidebar
</aside>

<!-- Footer section -->
<footer>
  Website Footer
</footer>

<!-- Containers -->

<!-- Generic block container -->
<div>
  Block container
</div>

<!-- Generic inline container -->
<span>
  Inline text
</span>
```

CSS

```
internal:
<style>
  h1 {
    color:blue;
    padding-left 200px:
  }
</style>

<strong style="color:pink">xxx</strong>

<div style="color:pink;">
  <><>
</div>
```

```

external:
    <link rel="stylesheet" href="css/style.css">
    <div class="div1" >

    </div>
    <div class="div2" >

    </div>

in style css:
h1{
    color: red;
}
.div1{
    xxx
}
.div2{
    xxx
}

```

Spider n probe (scraper)

```

import requests
from bs4 import BeautifulSoup

def get_page(url):
    response = requests.get(url)

    soup = BeautifulSoup(response.content, 'html.parser')

    #print(soup.find("a"))
    #print(soup.find_all("a"))

    tag = soup.find_all("a")

    for i in tag:

```

```
url2 = tag.get("href")
print(url2)

get_page(input("what url would you like to scrape? "))
```

<https://chatgpt.com/share/6a913b65-6510-83e8-9616-414308db391c>

```
import requests
from bs4 import BeautifulSoup
from urllib.parse import urljoin, urlparse

# 已经访问过的 URL
visited_urls = set()

def spider_urls(url, keyword):
    # 如果 URL 已经访问过, 就不再访问
    if url in visited_urls:
        return

    visited_urls.add(url)

    try:
        # 发送 GET 请求
        response = requests.get(
            url,
            timeout=5,
            headers={
                "User-Agent": "Mozilla/5.0"
            }
        )

    except requests.RequestException as e:
        print(f"Request failed: {url}")
        return

    # 只处理正常响应
    if response.status_code != 200:
        return
```

```

# 解析 HTML
soup = BeautifulSoup(response.text, "html.parser")

# 找出所有 <a> 标签
for tag in soup.find_all("a"):
    href = tag.get("href")

    # href 不存在或者为空
    if not href:
        continue

    # 把相对 URL 转换成完整 URL
    full_url = urljoin(url, href)

    # 只爬取同一个网站
    if urlparse(full_url).netloc != urlparse(url).netloc:
        continue

    # 如果关键词存在于 URL 中，就输出
    if keyword in full_url:
        print(f"[+] Found: {full_url}")

    # 继续递归爬取
    spider_urls(full_url, keyword)

# 输入起始 URL
url = input("Enter the URL you want to scrape: ")

# 输入关键词
keyword = input("Enter the keyword to search for in the URL: ")

# 开始爬取
spider_urls(url, keyword)

```

<https://chatgpt.com/share/6a92b157-59b8-83ee-bfc7-464b08742ab6>

Port Scanner

kali: nmap IP

how to utilize: <https://chatgpt.com/share/6a9704f5-6e64-83e8-8bf2-7c9d21036066>

if __name__ == '__main__':

<https://chatgpt.com/share/6a979c2f-ad90-83ee-b8dc-3f26a0736d0c>

```
app.main()
```

```
from flask import Flask

app = Flask(__name__)

@app.route('/')
def hello_world():
    return 'Hello world'

if __name__ == '__main__':
    app.run()
```

<https://chatgpt.com/share/6a97d75a-3af8-83ee-8fc2-d15dd59608b7>

Open Redirect

```
mysite.com/open?url=attacker.com
```

urlencode the blue part:(percent-encoding)

```
mysite.com/open?url=%75%72%6c%3d%61%74%74%61%63%6b%65%72%2e%63%6f%6d
```

for phishing attack.

different types: <https://chatgpt.com/share/6a9ab028-0c78-83ee-afdc-91ea19a7326d>

IDORS

Insecure Direct Object Reference

<https://chatgpt.com/share/6a9ab61a-79c8-83ee-9aa7-680cb3584525>

refresh and check network